

Ans (1)

Original array [20 33 41 55 56 60 61 70]

Quick sort call (QSC) [...] (low=0, high=7)

pivot 55: [...]

swapping 20 and 70 : [70 ... 20]

swapping 33 and 61 : [70 61 41 55 56 60 33 20]

swapping 41 and 60 : [70 61 60 55 56 41 33 20]

swapping 55 and 56 : [70 61 60 56 55 41 33 20]

partition complete : [70 61 60 56 55 41 33 20]

After partition [...] (pivot 56)

QSC [70 61 60] (low=0, high=2)

partitioning with pivot 61 : [70 61 60]

QSC [55 41 33 20] (low 4, high 7)

partitioning with pivot 41 : [55 41 33 20]

Q5c [33 20] (low 6; high 2)

partitioning with pivot 33 : [33 20]

Final Sorted Array [70, 64, 60, 56, 55, 41, 33, 20]

(1/2) 0 Ans 2

Best case $O(n)$ books are already sorted

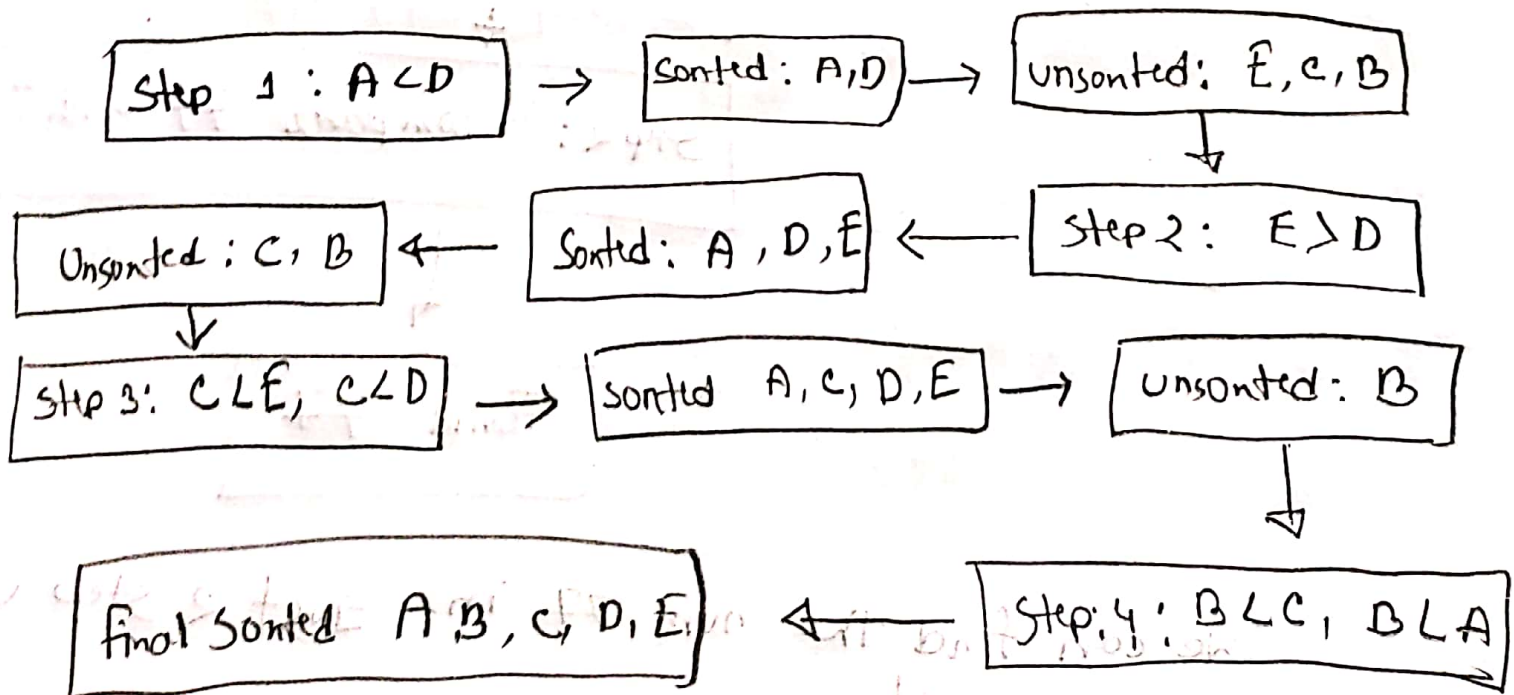
Worst case $O(n^2)$ books are reverse order

example:

assume we have books by first letter

[D, A, E, C, B]

Initial array (unsorted) D, A, E, C, B



Ans 3

We should choose Binary search as it is faster and efficient for sorted array. With time complexity $O(\log n)$

Within 3 steps:

[Initial array]

Step-1: compare 77 with 50

Right Half 50, 77, 88, 99

Step 2: compare 77 with 77

Found 77

We can find the num 77 in just 2 step using Binary search

9

Ans 4

Now-major formula

$$\text{loc}(A[i][j]) = \text{base_addr} + [(i - \text{lower_bound}_i) * n + (j - \text{lower_bound}_j)] * \text{size_of_element}$$

Here,

$$\text{loc}(A[5][15]) = 4500$$

$A[60][60]$ is double array

double data is 8 bytes

array start 0 (lower bound is 0)

And the base add of the array

$A[5][15]$:

$$4500 = \text{base_addr} + [(5-0) * 60 + (15-0)] * 8$$

$$\Rightarrow 4500 = \text{base_addr} [315] * 8$$

$$\therefore \text{base_addr} = 4500 - 2520 = 1980$$

memory loc of A[15][45]

$$= 1980 + [(15-0) * 60 + (45-0)] * 8$$

$$= 1980 + [900 + 45] * 8$$

$$= 1980 + 945 * 8$$

$$= 9540$$

So the memory location of A[15][45] is 9540.

And the base add of the array is 1980.

A[15][45]

$$1980 + 945 * 8 = 9540$$

$$9540 - 1980 = 7560$$

$$7560 / 8 = 945$$

Ans 50

Time complexity:

Outer loop runs $(n-1)$ times

For each iteration of the outer loop, inner

loop makes $(n-j)$ comparisons

In the worst case, $(1-n)$

① When $j=1$, there are comparisons $(n-1)$

② When $j=2$, there are $(n-2)$ comparisons

③ This pattern continues until $j=n-1$

where there is 1 comparison

$$(n-1) + (n-2) + \dots + 1$$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2}$$

So the time complexity will be $O(n^2)$

Q 2 sol

Q

Ans 5 (u)

Base case: $(n == 1)$ return 1 without making any recursive call

① In the depth of the recursive tree is $(n-1)$. It calls itself $n-1$ times

② At each level it performs constant time

So the function complexity $O(n)$

$$T(n) = T(n-1) + (1-1)$$

$$T(n) = T(n-1) + 1$$

$$T(n) = T(n-1) + 1$$

$$T(n) = T(n-1) + 1$$

$$T(n) = T(n-1) + 1$$

① Ans 16 ①

$$f(n) = \frac{1}{3}(n^3 - 5n^2) \text{ is } \Omega(n^3)$$

$f(n)$ is $\frac{1}{3}n^3$, and since the cubic term grows faster than the quadratic term $-5n^2$ so,

$$f(n) = \frac{1}{3}n^3 - 5n^2 > \frac{1}{6}n^3 \text{ (n large)}$$

Therefore, we can choose $c = \frac{1}{6}$ and prove that
the large enough n , $f(n) > c \cdot n^3$

So $f(n)$ is $\Omega(n^3)$

Q. 2 Ans 6 (ii)

$f(n) = 5n^2 - 3$ is $O(n^2)$ to prove that, some

constant $c > 0$ and n_0 , $f(n) \leq c \cdot n^2$ for all

$n \geq n_0$

for large n , the constant term -3 becomes insignificant compared to $5n^2$. Therefore,

$$f(n) = 5n^2 - 3 \leq 6n^2 \quad (\text{here large } n)$$

$c = 6$ and conclude that $f(n) \leq c \cdot n^2$ for large n .

So, $f(n)$ is $O(n^2)$

10

Ans 6 (iii)

$f(n) = \frac{1}{2}n^2 - 7n$ is $\Theta(n^2)$ to prove that
we need to show both

$f(n)$ is $O(n^2)$ and $f(n)$ is $\Omega(n^2)$

$O(n^2)$: large n , the linear term $-7n$ become
insignificant compared to $\frac{1}{2}n^2$ So,

$$f(n) = \frac{1}{2}n^2 - 7n \leq n^2 \quad (\text{large enough } n)$$

$c = 1$ and therefore $f(n) \leq c \cdot n^2$

$\Omega(n^2)$: for large n , the linear term $-7n$ is domin-
ated by $\frac{1}{2}n^2$:

$$f(n) = \frac{1}{2}n^2 - 7n \geq \frac{1}{3}n^2 \quad (\text{large } n)$$

therefore $c = \frac{1}{3}$ and prove that $f(n) \geq c \cdot n^2$

Since both $O(n^2)$ and $\Omega(n^2)$ we can conclude that

$$f(n) = \Theta(n^2)$$